

Privacy-Preserving Batch Oracles: Hierarchical Proofs via Accumulate and zkSTARK

Annabelle Shalom, ChristyAna Viva, Paul Snow

Submission Track: Science of Blockchain Conference (SBC) 2026 — March 2026

Keywords: zkSTARK, privacy-preserving oracles, Accumulate, StarkNet, hierarchical proofs, selective disclosure, decentralized identity, authority provenance, temporal integrity

Abstract

Oracle systems face a fundamental tension: data must be verifiable on-chain, yet revealing raw data enables front-running, privacy violations, and competitive disadvantage. We present a two-layer architecture combining Accumulate’s hierarchical batch proofs with zkSTARK’s zero-knowledge computation. The first layer uses Accumulate to commit data off-chain and generate compact receipts proving data existence, with Bitcoin anchoring for security. Accumulate’s URL-based addressing (`acc://authority/path`) cryptographically binds data to controlling identities, enabling authority verification without identity disclosure. The second layer uses Cairo/zkSTARK to prove properties of committed data without revealing values. This achieves: (1) batch proof efficiency—a single receipt proves unlimited data entries with $O(\log N)$ verification; (2) hierarchical trust—a single Bitcoin anchor validates unlimited receipts; (3) temporal integrity—Bitcoin-anchored timestamps prove when data existed, critical in an era where AI can fabricate any unanchored record; (4) authority provenance—prove which entity authorized data without revealing its real-world identity; (5) selective disclosure—prove predicates (“balance > threshold”) without revealing inputs. We present a Cairo reference implementation and identify deployment requirements for StarkNet.

1 Introduction

Decentralized oracles secure over \$20 billion in DeFi protocols [7], yet face three persistent challenges:

Efficiency: Traditional oracles prove each data point individually, scaling linearly with data volume. For high-frequency feeds (prices, IoT, cross-chain state), verification costs become prohibitive.

Privacy: Raw oracle data enables exploitation. Price feeds enable front-running. Identity attestations leak personal information. Business data reveals competitive intelligence.

Temporal Integrity: In an era of AI-generated content, proving *when* data existed and *who* authored it becomes critical. The past begins today—records created now become tomorrow’s historical evidence. Without cryptographic timestamps and authority binding, any document, image, or dataset can be retroactively fabricated or disputed. Privacy compounds this protection: AI cannot fake what it cannot see. Committed but unrevealed data is immune to bot-driven front-running and preemptive fabrication.

Existing solutions address these independently. Batch proving (rollups) improves efficiency but exposes data. Zero-knowledge proofs provide privacy but require per-proof computation. No system achieves all three simultaneously with practical efficiency.

1.1 Contribution

We present a two-layer architecture:

Layer 1: Accumulate Batch Proofs

- Data is committed to Accumulate Network chains via URL-addressed accounts
- URL structure (`acc://authority/path`) cryptographically binds data to ADI authorities
- Stateful Merkle Tree (SMT) with Bitcoin anchoring
- A single receipt proves an entire data chain ($O(\log N)$ hashes)
- A single anchor validates unlimited receipts

Layer 2: zkSTARK Selective Disclosure

- Cairo contracts verify Accumulate receipts on StarkNet
- zkSTARK proofs demonstrate properties of committed data
- Zero-knowledge: verifier learns predicate result, not input values

Combined properties:

Property	Mechanism	Benefit
Batch efficiency	Hierarchical receipts	99%+ cost reduction
Trust minimization	Bitcoin anchoring	PoW security
Temporal integrity	Anchored timestamps	Prove when, immutably
Authority provenance	URL/ADI binding	Prove who, not identity
Selective disclosure	zkSTARK predicates	Prove what, not values

1.2 Paper Organization

Section 2 covers background on Accumulate and zkSTARK. Section 3 presents the hierarchical proof architecture. Section 4 details the privacy layer. Section 5 provides the Cairo implementation. Section 6 analyzes security properties. Section 7 discusses deployment requirements, open problems, and future research directions.

2 Background

2.1 Accumulate Protocol

Accumulate is a delegated proof-of-stake network providing identity-based data management with Bitcoin anchoring [1]. Key features:

Accumulate URLs: All accounts are addressed via URLs following the pattern `acc://[identity]/[path]`. Examples:

<code>acc://oracle.acme</code>	# ADI (identity root)
<code>acc://oracle.acme/prices</code>	# Sub-account for price data
<code>acc://oracle.acme/prices/btc-usd</code>	# Specific price feed
<code>acc://gov.state/credentials/citizens</code>	# Govt credential

URLs are not merely paths—they encode cryptographic authority. The identity at the root (`acc://oracle.acme`) controls all sub-paths.

Accumulate Digital Identifiers (ADI): The identity portion of a URL is an ADI with associated key hierarchies. ADIs can define multiple key pages with different signing thresholds, delegate authority to sub-identities, and control all accounts and chains under their URL path.

Authority Binding: Any data at `acc://oracle.acme/prices/*` is cryptographically controlled by the `acc://oracle.acme` ADI. A receipt proving data exists at a URL implicitly proves which authority created it—without revealing the authority’s real-world identity if pseudonymous.

Data Chains: Each URL path can have multiple chains storing arbitrary data entries. Chains are append-only Stateful Merkle Trees.

Receipts: Compact proofs that a specific entry exists in a chain, verifiable against a root anchor. Receipt size is $O(\log N)$ for N entries. Receipts include the full URL path, binding data to authority.

Multi-Protocol Anchoring: Accumulate periodically commits network state to multiple protocols: Bitcoin (proof-of-work finality), smart contract chains (Ethereum, StarkNet, etc.), and future protocols accepting anchors. This enables receipts to be verified against anchors stored on whichever chain the application uses.

Abstract Account Security: Beyond data attestation, Accumulate enables a paradigm where data entries authorize actions and anchor proofs replace signatures:

Traditional Model	Accumulate Model
Private key signs tx	Data entry exists in chain
Signature verification	Receipt verification
Key compromise = full access	Entry-level authorization
Revocation requires new keys	Remove entry = revoke

2.2 Hierarchical Receipt Structure

Accumulate receipts have the structure:

```
Receipt:
  start:      Hash256      // Entry being proven
  start_index: u64         // Position in chain
  end:        Hash256      // Entry at anchor point
  end_index:   u64         // Anchor entry position
  anchor:      Hash256      // SMT root at anchor point
  entries:     []Entry      // Path from start to anchor
    - right:   bool        // Direction flag
    - hash:    Hash256      // Sibling hash
```

Verification: Walk path from `start` to computed root, compare with `anchor`.

2.3 zkSTARK Fundamentals

Scalable Transparent Arguments of Knowledge [2] provide:

- **Transparency:** No trusted setup; randomness from public transcript
- **Zero-knowledge:** The verifier learns the statement's truth, not witness values
- **Scalability:** Prover $O(N \log N)$, verifier $O(\log^2 N)$
- **Post-quantum:** Based on hash functions, not elliptic curves

Cairo [5] compiles to AIR (Algebraic Intermediate Representation) for STARK proving.

2.4 Related Work

Chainlink [10]: Attestation-based oracles. No batch efficiency or privacy.

DECO [3]: TLS-based private attestation. Proves web data without revealing content. Single-source, no batching.

Town Crier [4]: TEE-based oracle privacy. Hardware trust assumption.

Mina [9]: Recursive SNARK state proofs. Different trust model (no Bitcoin anchor).

Our work combines Accumulate's batch efficiency with zkSTARK privacy, avoiding trusted hardware and setup ceremonies.

3 Hierarchical Proof Architecture

3.1 Trust Hierarchy

```
Anchor Targets (one or more)
|
+-- Bitcoin ----- PoW finality
+-- Ethereum ----- PoS finality
+-- StarkNet ----- Direct L2 access
|
+-- Accumulate Anchor (verified once per target)
    |
    +-- acc://oracle.acme          (ADI authority)
    |   +-- /prices/btc-usd -- 10K entries
    |   +-- /prices/eth-usd -- 10K entries
    |
    +-- acc://gov.state           (ADI authority)
    |   +-- /credentials --- 1M credentials
    |
    +-- acc://bank.example        (ADI authority)
    |   +-- /accounts ----- attestations
```

Trust propagation:

1. The anchor target provides finality guarantees
2. The Accumulate anchor is committed to one or more targets (one verification per target)
3. Receipts verify against the stored anchor (no re-verification needed)
4. The URL path binds data to the controlling ADI (authority provenance)
5. Entries are proven via a receipt (no per-entry cost)

3.2 Authority Provenance

The URL structure provides cryptographic authority binding without trusted registries:

What Receipt Proves	What URL Proves
Data exists in Accumulate	Which ADI authorized the data
Data is Bitcoin-anchored	Authority hierarchy
Data hasn't been modified	Pseudonymous authority

Example: A receipt for `acc://oracle.acme/prices/btc-usd` proves:

- The price data exists and is anchored to Bitcoin
- The `acc://oracle.acme` ADI authorized this data
- No one without `oracle.acme` keys could have written it

3.3 Efficiency Analysis

Operation	Verification Cost	Frequency
Anchor registration	$O(1)$	Per anchor period
Receipt verification	$O(\log N)$ SHA-256	Per chain update
Entry membership	$O(1)$ lookup	Per query

Comparison with linear verification:

Data Volume	Linear Proofs	Hierarchical
100 entries $\times$ 10 chains	1,000	1 anchor + 10 receipts
10K entries $\times$ 100 chains	1,000,000	1 anchor + 100 receipts
1M entries $\times$ 1K chains	1,000,000,000	1 anchor + 1K receipts

4 Privacy Layer: Selective Disclosure

4.1 Problem Statement

Proving data exists (via receipt) is necessary but insufficient. Applications need to prove *properties* of data without revealing *values*.

Examples:

- Credit: "Income $>$ \$50,000" without revealing exact income
- Trading: "Price moved $>$ 1%" without revealing price
- Identity: "Age $\geq$ 18" without revealing birthdate
- Compliance: "Passed sanctions check" without revealing identity

4.2 Commitment-Based Privacy

Data is stored in Accumulate as commitments, not plaintext:

- `commitment = Hash(value||blinding_factor)`

The receipt proves the commitment exists. A separate zkSTARK proves knowledge of the preimage satisfying some predicate.

Workflow:

1. **Commit:** Store `commitment = Hash(value||r)` in Accumulate chain
2. **Prove existence:** The Accumulate receipt proves the commitment is anchored
3. **Prove property:** zkSTARK proves "I know (value, r) such that `Hash(value || r) = commitment` AND `predicate(value) = true`"

4.3 Privacy Use Cases

The combination of Accumulate URLs and zkSTARK proofs enables three types of privacy:

1. **Value privacy:** Prove properties without revealing values (zkSTARK)
2. **Authority verification:** Prove who authorized data without revealing identity (URL + pseudonymous ADI)
3. **Selective disclosure:** Prove membership without revealing which entry

4.3.1 Private Credit Scoring

Scenario: A DeFi lending protocol needs to verify borrower creditworthiness without accessing raw financial data.

Flow:

1. Credit bureau commits score: $\text{commitment} = \text{Hash}(\text{score}||r)$
2. Commitment stored at `acc://equifax.bureau/scores/{hash(user_pubkey)}`
3. The receipt proves: the commitment exists AND was written by `acc://equifax.bureau`
4. zkSTARK proves: “score > 650” without revealing score

The verifier learns: Score exceeds threshold; Equifax authorized the score (from URL authority).

The verifier does NOT learn: Actual score value; user’s real identity; other scores in the bureau.

4.3.2 Front-Running Resistant Price Feeds

Scenario: A DEX needs price data but raw prices enable sandwich attacks.

Flow:

1. Oracle commits prices: $\text{commitment} = \text{Hash}(\text{price}||\text{timestamp}||r)$
2. Commitments appended to `acc://chainlink.oracle/feeds/btc-usd`
3. During time T: The receipt proves the commitment exists under a known authority
4. zkSTARK proves: “price $\in$ [low, high]” without exact value
5. After time $T+\Delta$: Reveal exact price (front-running window passed)

4.3.3 Private Identity Attestation

Scenario: A service requires age verification without collecting birthdate.

Flow:

1. DMV issues credential: $\text{commitment} = \text{Hash}(\text{birthdate}||\text{license_class}||r)$
2. Credential stored at `acc://dmv.california/credentials/{hash(citizen_pubkey)}`
3. The receipt proves the credential exists under a government authority
4. zkSTARK proves: “birthdate implies age ≥ 21 ”

4.3.4 Multi-Authority Credentials

Scenario: A loan application requires proof of employment AND credit score from different authorities.

Flow:

1. User obtains receipts from both URLs
2. Both receipts verify against the same Accumulate anchor (single anchor verification)
3. zkSTARK proves: “employed = true AND score > 700”

Efficiency: Two authorities, two receipts, one anchor, one STARK proof.

4.4 Private Authorization via Anchor Proofs

Traditional authorization uses signatures: “This transaction is valid because key K signed it.” Accumulate enables: “This action is authorized because entry E exists in chain C.”

Combined with zkSTARK: “This action is authorized because *some* entry exists satisfying predicate P”—without revealing which entry.

Applications:

Use Case	Auth Entry	zkSTARK Predicate
Building access	Employee cred	department = X
API rate limiting	Subscription tier	tier $\geq$ ‘pro’
Voting	Voter registration	registered AND NOT voted
Trading	KYC attestation	kyc_passed AND ok_jurisdiction

4.5 Composing Receipts and STARKs

Full verification requires both layers:

```
/// Combined verification: existence + property
fn verify_private_oracle(
    receipt: Receipt,
    anchor_id: felt252,
    threshold: u128,
    // Private witness:
    value: u128,
    blinding: Hash256,
) -> bool {
    // Layer 1: Verify receipt
    if !verify_receipt(receipt, anchor_id) {
        return false;
    }
    // Layer 2: Verify property
    verify_threshold_private(
        receipt.start, threshold, value, blinding
    )
}
```

The verifier receives: Receipt, anchor_id, threshold, STARK proof (all public). The verifier does NOT receive: value, blinding (private witness).

5 Implementation

5.1 Contract Architecture

```
AccumulateVerifier Contract
Storage:
  anchors: Map<felt252, AnchorInfo>
  proven_chains: Map<felt252, bool>
Functions:
  register_anchor(id, hash, bitcoin_block)
  verify_receipt(receipt, anchor_id) -> bool
  verify_and_store(receipt, anchor_id, chain_id)

PrivateOracle Contract
Functions:
  verify_threshold(receipt, anchor_id, thresh) -> bool
  verify_range(receipt, anchor_id, low, high) -> bool
  verify_membership(receipt, anchor_id, root) -> bool
```

5.2 Gas Estimates

Preliminary estimates, pending StarkNet testnet benchmarks:¹

Operation	SHA-256 Calls	Est. Gas
register_anchor	0	~50,000
verify_receipt (20 levels)	20	~200K–400K
threshold_check (private)	1 + STARK	~300K–500K
combined verification	21	~500K–900K

5.3 Reference Implementation

Full Cairo source is provided in Appendix A. Key components:

- **AccumulateVerifier**: Receipt verification against stored anchors
- **PrivateThreshold**: Commitment verification with threshold predicate
- **PrivateRange**: Range proof on committed value
- **PrivateMembership**: Set membership proof

¹SHA-256 is more expensive than Pedersen in Cairo. Actual costs depend on StarkNet fee market and SHARP batching.

6 Security Analysis

Our construction does not introduce novel cryptographic primitives. Security reduces to established results for Merkle trees [11], STARK proofs [2], and digital signatures [12]. We cite these foundations and prove only the composition.

6.1 Threat Model

System model:

- Accumulate Network: DPoS network anchoring to Bitcoin/Ethereum/StarkNet
- Anchor: Merkle root of Accumulate state, committed to target chain
- Receipt: Merkle proof from entry to anchor
- Verifier: Cairo contract on StarkNet

Adversary capabilities: Controls a minority of Accumulate validators ($< 1/3$); observes all on-chain transactions; submits arbitrary receipts and proofs.

Adversary limitations: Cannot break SHA-256 collision resistance; cannot break STARK soundness (2^{-128}); cannot forge Ed25519 signatures.

6.2 Security Foundations

Theorem 1 (Merkle Security [11]): For a Merkle tree with collision-resistant hash H , no PPT adversary can produce a valid proof for an element not in the tree except with probability $\leq \text{Adv}_H^{\text{CR}}$.

Application: Accumulate receipts are Merkle proofs using SHA-256. Receipt soundness follows directly.²

Theorem 2 (STARK Soundness [2]): No PPT adversary can produce an accepting STARK proof for a false statement except with probability $2^{-\lambda}$.

Theorem 3 (STARK Zero-Knowledge [2]): A STARK verifier learns nothing beyond the statement’s truth.

Theorem 4 (Commitment Binding): SHA-256 commitment $c = H(v\|r)$ cannot be opened to $v' \neq v$ except with probability $\leq \text{Adv}_{\text{SHA256}}^{\text{CR}}$.

Proof: If an adversary opens c to both (v, r) and (v', r') , then $H(v\|r) = H(v'\|r')$ is a collision. $\square$

6.3 Authority Binding

Theorem 5: Data at URL `acc://adi/path` can only be written by keys authorized by ADI `acc://adi`.

Proof: By the Accumulate protocol specification [1]: (1) Each ADI has key pages defining authorized signers; (2) Transactions modifying `acc://adi/*` require a valid signature; (3) Ed25519 is EUF-CMA secure [12]; (4) Unauthorized write requires signature forgery. Security reduces to Ed25519 unforgeability. $\square$

Corollary: An ADI can be pseudonymous. Authority is verifiable without real-world identity linkage.

6.4 Composition

Theorem 6: Verifying an Accumulate receipt then proving a predicate via STARK preserves soundness and zero-knowledge.

Proof: Let Layer 1 = Receipt verification (public, deterministic); Layer 2 = STARK proof of $P(\text{value})$ given commitment.

Soundness: An adversary must either: forge a receipt (violates Theorem 1), or produce a STARK proof for false P (violates Theorem 2). By union bound: $\text{Pr}[\text{break}] \leq \text{Adv}_{\text{SHA256}}^{\text{CR}} + 2^{-\lambda}$.

Zero-Knowledge: Layer 1 is public; Layer 2 reveals only $P(\text{value})$, not value (Theorem 3). Layers compose: public preprocessing + ZK proof is standard [2]. No information leakage between layers. $\square$

²Accumulate uses a Stateful Merkle Tree (SMT) [1] rather than Merkle Mountain Range (MMR) [13]. Unlike MMR, which maintains multiple “peaks” requiring sequential hashing to combine, SMT is a single continuous tree with anchors at block boundaries—no peak-bagging required.

6.5 Security Summary

Property	Proof Method	Citation
Receipt soundness	Merkle tree security	[11, 1]
Predicate soundness	STARK soundness	[2]
Zero-knowledge	STARK ZK	[2]
Commitment binding	Collision resistance	Standard
Authority binding	EUF-CMA signatures	[12]
Composition	Theorem 6 above	This work

Conclusion: No novel cryptography. Security derives from standard primitives. The contribution is architectural integration.

6.6 Limitations

Data availability: Accumulate must remain available for receipt generation. Proofs don't provide data recovery.

Anchor trust: Anchor registration currently requires a trusted operator. Future work: Bitcoin SPV verification in Cairo.

Commitment scheme: SHA-256 commitments are non-algebraic. Pedersen commitments would enable homomorphic proofs.

7 Discussion

7.1 Deployment Requirements

Immediate (Phase 1): Cairo contract deployment on StarkNet Sepolia; test vector generation from Accumulate testnet; basic receipt verification benchmarks.

Near-term (Phase 2): Accumulate SDK integration for receipt generation; private predicate library; gas optimization.

Future (Phase 3): Bitcoin SPV verification in Cairo (trustless anchors); recursive proof aggregation (SHARP); Pedersen commitments for homomorphic proofs.

7.2 Open Problems

1. **Optimal commitment scheme:** SHA-256 is simple but non-algebraic. Pedersen enables addition/multiplication proofs. Poseidon is STARK-friendly but less standard.
2. **Proof aggregation:** Can multiple private oracle queries batch into a single STARK?
3. **Data availability incentives:** Who stores raw data, and why?
4. **Cross-chain privacy:** Proving properties of data on Chain A to Chain B without revealing data to either.

7.3 Future Directions

Multi-Protocol Anchoring: Accumulate supports anchoring to multiple protocols simultaneously. Applications verify receipts against a local anchor (no bridge required). Each chain has native access to Accumulate proofs.

Bitcoin-Native Verification via BitVMX: BitVMX [8] enables RISC-V program verification on Bitcoin through fraud proofs. Future work could implement Accumulate receipt verification directly in BitVMX.

Private Credential Ecosystems: Employment/income verification, freelancer reputation, undercollateralized DeFi lending—all enabled by combining multiple ADI attestations with compound zkSTARK predicates.

AI and Machine Learning Integration: Model output attestation, training data provenance—as AI content proliferates, verifiable provenance becomes critical.

Recursive STARK Aggregation: StarkNet's SHARP enables recursive proof composition: batch 1000 private oracle queries into a single proof; target sub-cent verification cost per query.

7.4 Comparison Summary

System	Batch	Private	Authority	Trust
Chainlink	No	No	Reputation	Nodes
DECO	No	Yes	TLS cert	TLS/PKI
Town Crier	No	Yes	TEE	Intel SGX
Mina	Yes	Yes	None	Recursive SNARK
This work	Yes	Yes	URL/ADI	Bitcoin PoW

Authority provenance answers: “Who authorized this data?” Accumulate URLs cryptographically bind data to ADI authorities without requiring real-world identity disclosure.

8 Conclusion

We presented a two-layer architecture for privacy-preserving batch oracles achieving four properties not previously combined:

1. **Batch efficiency:** Accumulate’s Stateful Merkle Trees enable one anchor to validate unlimited data chains, each receipt proving unlimited entries. Verification costs reduce by 99%+ compared to per-entry proofs.
2. **Trust minimization:** Multi-protocol anchoring (Bitcoin, Ethereum, StarkNet) provides finality guarantees without centralized intermediaries.
3. **Authority provenance:** Accumulate’s URL-based addressing cryptographically binds data to controlling ADIs. Verifiers learn which entity authorized data without requiring real-world identity disclosure.
4. **Selective disclosure:** zkSTARK proofs demonstrate properties of committed data without revealing values. Combined with anchor proofs replacing signatures, this enables private authorization where systems verify “someone with property P is authorized” without learning who.

This paper focuses on oracle applications, and a reference implementation in Cairo is provided for StarkNet deployment.

Beyond Oracles: Accountable Privacy in the Age of AI

The architecture addresses a broader crisis: the collapse of trust in digital records. Generative AI can now produce convincing fake documents, images, and communications indistinguishable from authentic artifacts. Any digital record without cryptographic provenance can be disputed as fabricated—or fabricated and claimed as authentic.

The past begins today. Every email, contract, sensor reading, and attestation created now becomes historical evidence tomorrow. Without Bitcoin-anchored timestamps, there is no proof that a record existed at a claimed time. Without authority binding, there is no proof of authorship. The architecture presented here provides:

- **Temporal proof:** Bitcoin anchors prove data existed before a specific block height—immutable under proof-of-work security assumptions
- **Authority proof:** URL/ADI binding proves which entity committed data, even if that entity is pseudonymous
- **Selective verification:** zkSTARK enables proving properties of historical records without revealing contents

Identity matters even when anonymous. A pseudonymous authority that consistently signs accurate data builds reputation over time. The system enables accountability without surveillance: proves *that* an authorized entity made a claim, verifies *what* properties the claim satisfies, without exposing *who* the entity is or *what* exact values were claimed.

Why both matter: Identity without privacy creates surveillance infrastructure—every attestation exposes the attester and the attested. Privacy without identity enables anonymous claims with no accountability—anyone can assert anything. Neither alone suffices. The marriage of Accumulate’s authority binding with zkSTARK’s selective disclosure creates *accountable privacy*: verifiable claims from identifiable (but potentially pseudonymous) authorities, revealing only what must be revealed. This is the foundation for systems where trust can be established without requiring exposure.

In the end, AI cannot fake what it cannot see.

References

- [1] Accumulate Network. (2022). Accumulate Protocol Whitepaper. <https://accumulatenetwork.io>
- [2] Ben-Sasson, E., et al. (2018). Scalable, transparent, and post-quantum secure computational integrity. *IACR Cryptology ePrint Archive*.
- [3] Zhang, F., et al. (2020). DECO: Liberating Web Data Using Decentralized Oracles. *ACM CCS*.
- [4] Zhang, F., et al. (2016). Town Crier: An Authenticated Data Feed for Smart Contracts. *ACM CCS*.
- [5] StarkWare. (2024). The Cairo Programming Language Book. <https://book.cairo-lang.org>
- [6] Cohen, B., & Pietrzak, K. (2019). The Chia Network Blockchain. *Chia Network*.
- [7] Bonneau, J., et al. (2015). SoK: Research Perspectives and Challenges for Bitcoin and Cryptocurrencies. *IEEE S&P*.
- [8] Lerner, S. D., et al. (2024). BitVMX: A CPU for Bitcoin. *RootstockLabs*.
- [9] Mina Foundation. (2020). Mina Protocol: A Succinct Blockchain. <https://minaprotocol.com>
- [10] Breidenbach, L., et al. (2021). Chainlink 2.0: Next Steps in the Evolution of Decentralized Oracle Networks.
- [11] Merkle, R. (1980). Protocols for Public Key Cryptosystems. *IEEE S&P*.
- [12] Bernstein, D. J., et al. (2012). High-speed high-security signatures. *J. Cryptographic Engineering*.
- [13] Todd, P. (2012, documented 2016). Making UTXO Set Growth Irrelevant With Low-Latency Delayed TXO Commitments. <https://petertodd.org/2016/delayed-txo-commitments>. MMR adopted by Grin/Mimblewimble (2016–2019), Polkadot BEEFY bridges (2024).
- [14] Cevallos, A. (2025). The Merkle Mountain Belt. *arXiv:2511.13582*.